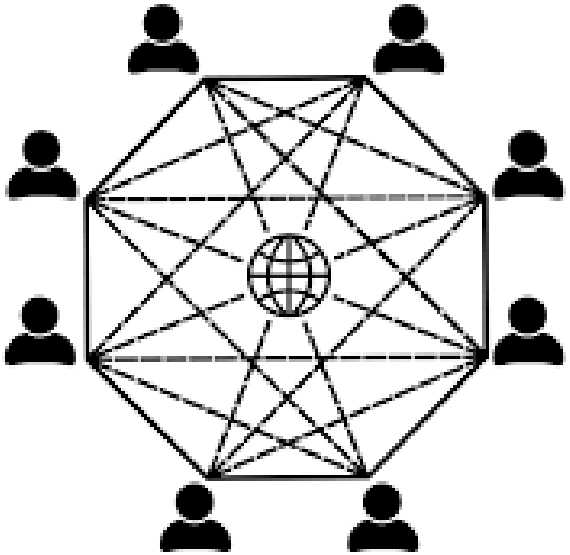
네트워크설계

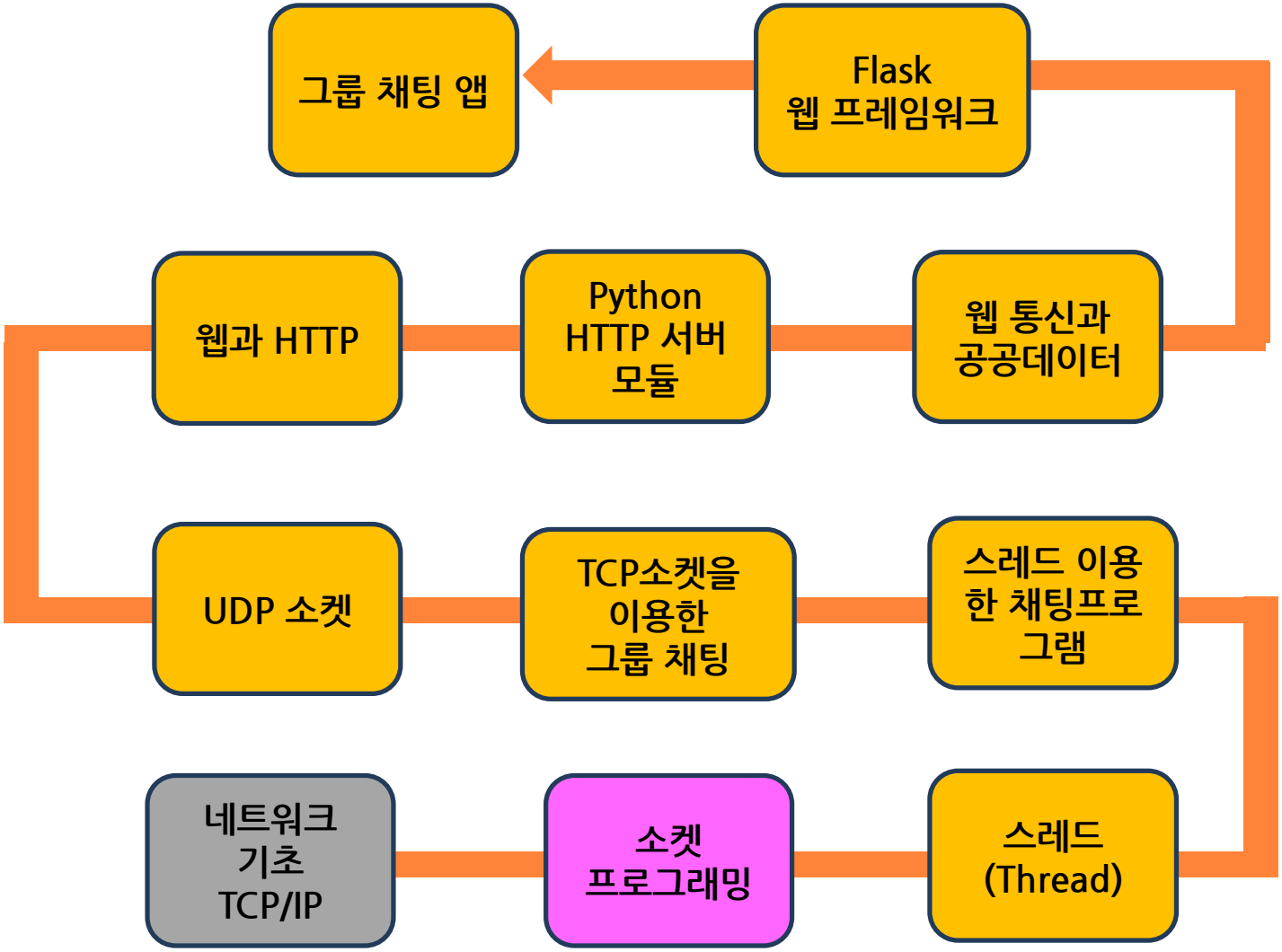
3주차 - 소켓 프로그래밍 응용



인공지능학과

그림참조: https://pxhere.com/ko/photo/1637436

3주차는 소켓 프로그래밍 응용



앱 제작 실습 병행

TCP/IP 4계층 구조

TCP/IP 4계층

① 응용 계층

② 전송 계층

③ 인터넷 계층

④ 링크 계층

① **응용 계층 : HTTP (웹)**
사용자 어플리케이션에서 사용하는 프로토콜
예) FTP, HTTP, SSH, Telnet, DNS, SMTP 등

② **전송계층 : TCP / UDP**
- 통신 노드 간의 연결 제어 및 자료 송수신을 담당
- TCP(Transmission Control Protocol) : 연결형, 신뢰성
- UDP(User Datagram Protocol): 비연결형, 비신뢰성

③ **네트워크 계층(인터넷 계층) : IP**
- 최종 목적지까지 정확하게 연결되도록 경로 설정: 라우팅
- Host를 구분하기 위해 주소를 할당: IP 주소
- IP(Internet Protocol) : 호스트들과 네트워크에서 주소 관리, 패킷을 라우팅

④ **링크 계층 : Ethernet / WiFi**
- 물리적으로 데이터가 네트워크를 통해 어떻게 전송되는지를 정의, 물리주소(MAC주소)를 사용

프로토콜과 인터페이스

- 프로토콜: 서로 다른 호스트에 위치한 동일 계층끼리의 통신 규칙
- 인터페이스: 같은 호스트에 위치한 상하위 계층 사이의 규칙
- 서비스: 하위 계층이 상위 계층에 제공하는 인터페이스

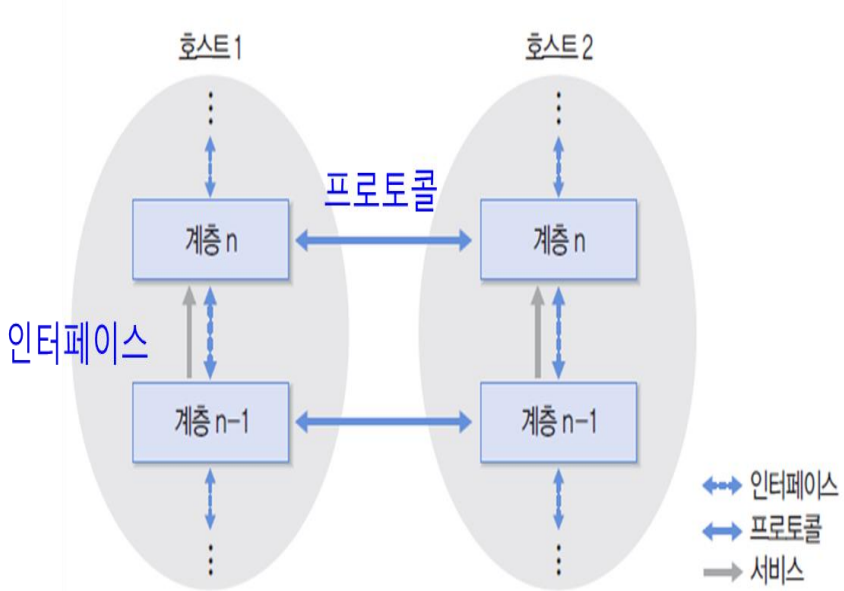
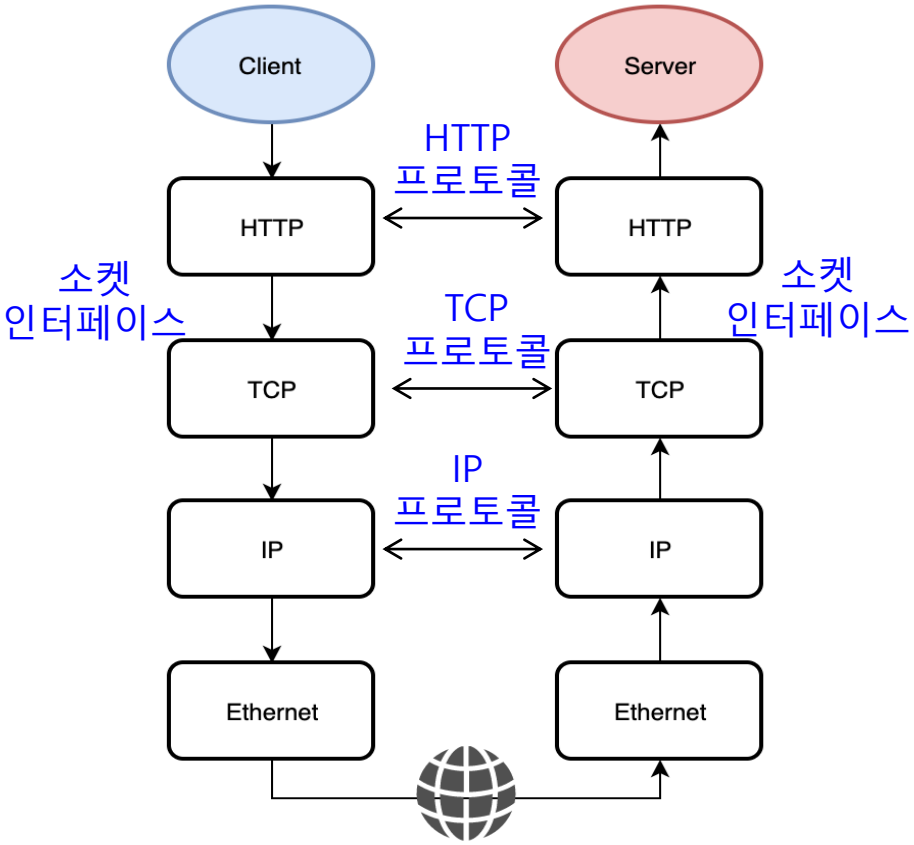
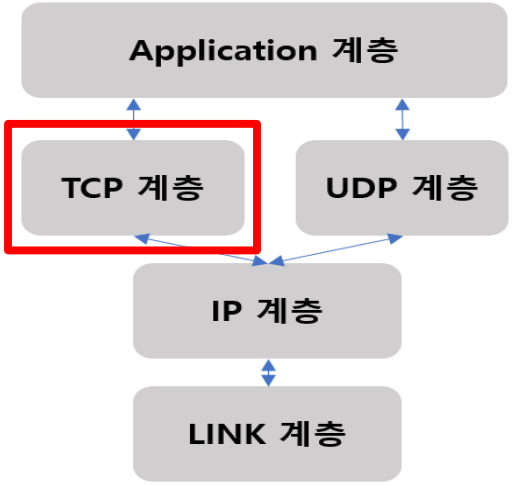


그림 1-5 인터페이스와 프로토콜



TCP(Transfer Control Protocol)



- **연결형**이며, **신뢰성**있는 데이터 통신을 가능하게 해주는 전송계층의 프로토콜

① **연결 지향적**(Connection-Oriented)

- 데이터를 전송하기 전에 먼저 수신 측과 연결을 설정

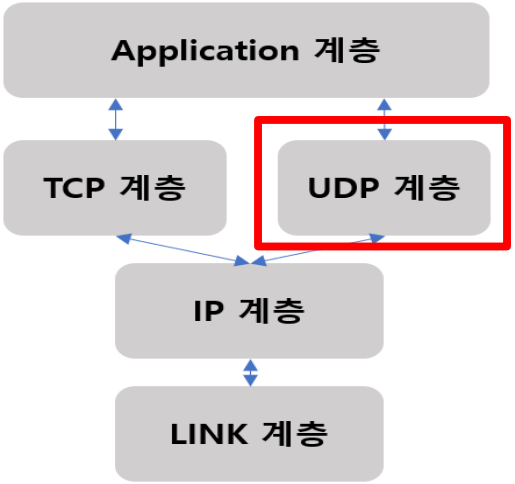
② **신뢰성 있는 데이터 통신**

- TCP는 송신 측과 수신 측 사이의 흐름제어 / 오류제어 / 혼잡제어 기능 제공

- 전송되는 데이터의 양을 조절하는 **흐름 제어**
- 데이터가 유실되거나 잘못된 데이터가 수신되었을 경우 대처하는 방법인 **오류 제어**
- 네트워크 혼잡에 대처하는 **혼잡 제어**

→ **응용계층은 전송계층의** TCP 덕분에 예외 상황에 크게 신경 쓰지 않고 응용계층 동작에 집중할 수 있음

UDP(User Datagram Protocol)



- **비연결형, 비신뢰성 서비스를 제공하는 전송계층 프로토콜**

- 흐름제어 기능이 없어 버퍼 오버플로Buffer Overflow에 의한 데이터 분실오류가 발생할 수 있음
- 에러제어 기능은 없으며 최소한의 오류검출 기능만 제공
  (패킷에러가 검출되면 폐기: 재전송하지 않음)

- **TCP보다 속도가 빠르다**
- 음성 및 비디오를 위한 실시간 스트리밍 서비스에 주로 사용 (그 외 DNS, DHCP등)

IP 주소 (IPv4 주소)

- 네트워크상에서 **컴퓨터(Host)를 식별하기 위해 부여된 주소**
- **32bit(4byte)** 길이로 구성된 논리적인 주소체계 (2^{32} 개 IP주소 배정가능)
- IP 주소는 **네트워크 주소**와 **호스트 주소** 두 부분으로 나뉜다. (IP주소 배정 규칙)

IP 주소 = Network ID + Host ID

	Network ID			Host ID
IPv4 주소	**192 .**	**168 .**	**10.**	**10**
서브넷 마스크	**255.**	**255.**	**255.**	**0**
	11111111	**11111111**	**11111111**	**00000000**

** 서브넷 마스크(AND 연산) 은 IP주소에서 네트워크 부분과 호스트 부분을 나누는 역할*

- Network(네트워크) ID : 네트워크 주소 (Prefix)
- Host(호스트) ID: 하나의 네트워크 내에 존재하는 호스트를 구분하기 위한 주소

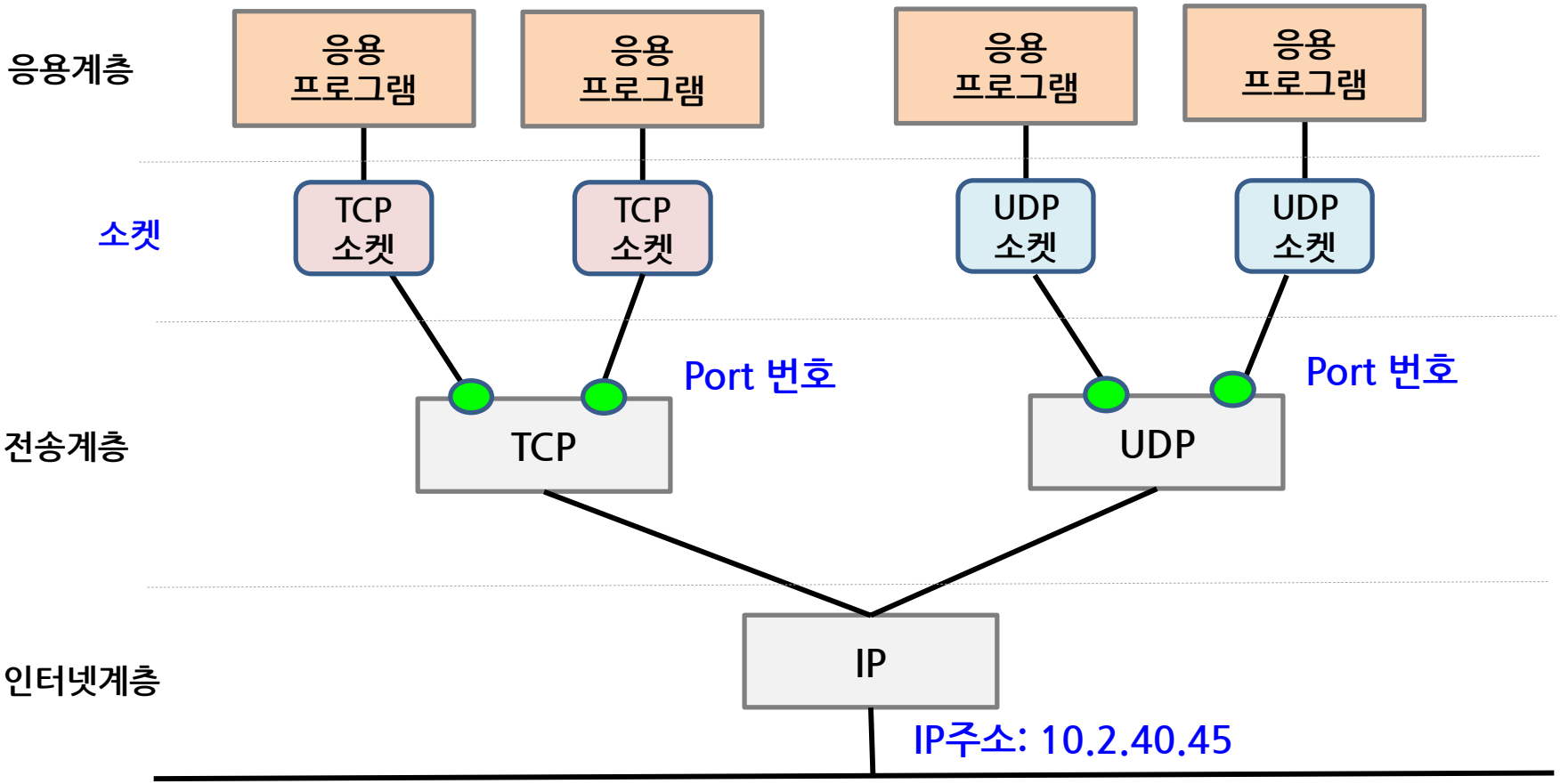
IP주소의 종류

종류	특징
공인 IP (Public IP)	• **전세계적으로 중복되지 않는 유일한 주소** • 인터넷 서비스 제공업체(ISP)가 사용자에게 할당하는 주소로 **인터넷 가능**
사설 IP (Private IP)	• **IP주소 부족문제를 해결하기 위해 등장** • 일반 가정이나 회사 등의 내부 네트워크에 할당된 IP 주소 • **사설 IP 주소만으로는 인터넷에 직접 연결할 수 없음** • 내부에서 외부로는 접근이 가능하지만 외부에서는 내부로 접근이 불가능 • IPv4 사설 IP 대역 - **10**.0.0.0 ~ **10**.255.255.255 (10.0.0.0/8) - **172.16.**0.0 ~ 172.31.255.255 (172.16.0.0/12) - **192.168**.0.0 ~ 192.168.255.255 (192.168.0.0/16)
루프백 IP (localhost)	• 네트워크상에서 자신을 나타내는 가상적인 주소 • **외부 네트워크를 사용하지 않고, 로컬에서 소프트웨어를 테스트하는 데 사용** • 대표적인 루프백 IP는 127.0.0.1 (localhost)를 사용
브로드캐스트 IP	• **네트워크에 있는 컴퓨터나 장비 모두에게 한 번에 데이터를 전송하는데 사용** • 255.255.255.255 또는 • IP 주소의 SUBNET 부분을 제외한 호스트 자리를 2진수 1로 채운 것
모든 IP	• 0.0.0.0**(모든 IP 주소)'는 내 컴퓨터에 연결된 모든 주소**

2주차 소켓 프로그램 정리

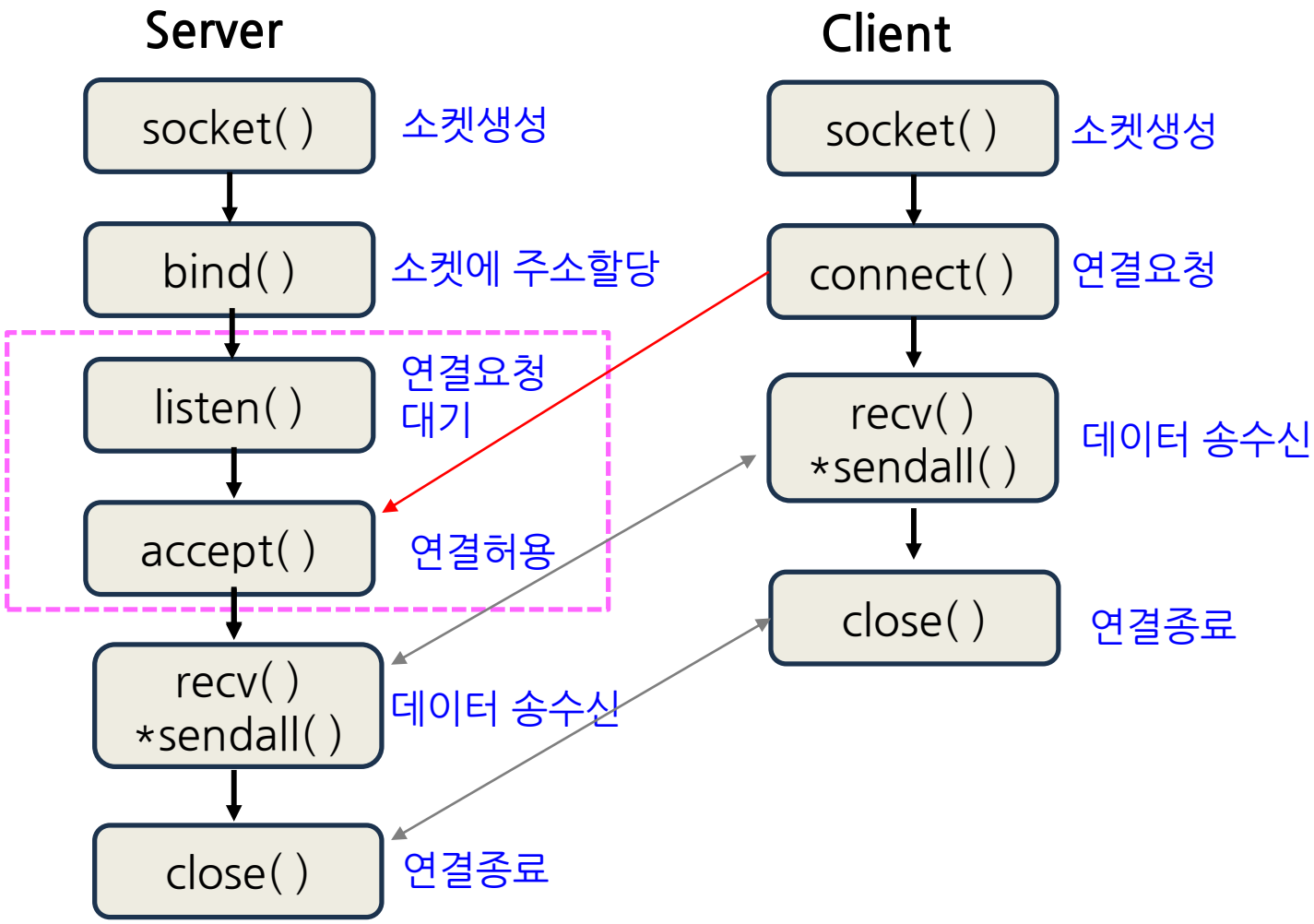
■ 네트워크 소켓

- 응용 프로그램간(서버와 클라이언트) 네트워크를 통해 서로 통신할 수 있도록 연결해 주는 소프트웨어(라이브러리)
- 응용계층과 전송계층 사이에 위치, 응용프로그램이 TCP/IP를 이용할 수 있도록 인터페이스 역할
- 소켓에는 IP 주소와 Port번호가 할당 (IP주소로 컴퓨터를 특정, Port번호로 프로세스를 특정)



■ 소켓 프로그래밍 절차

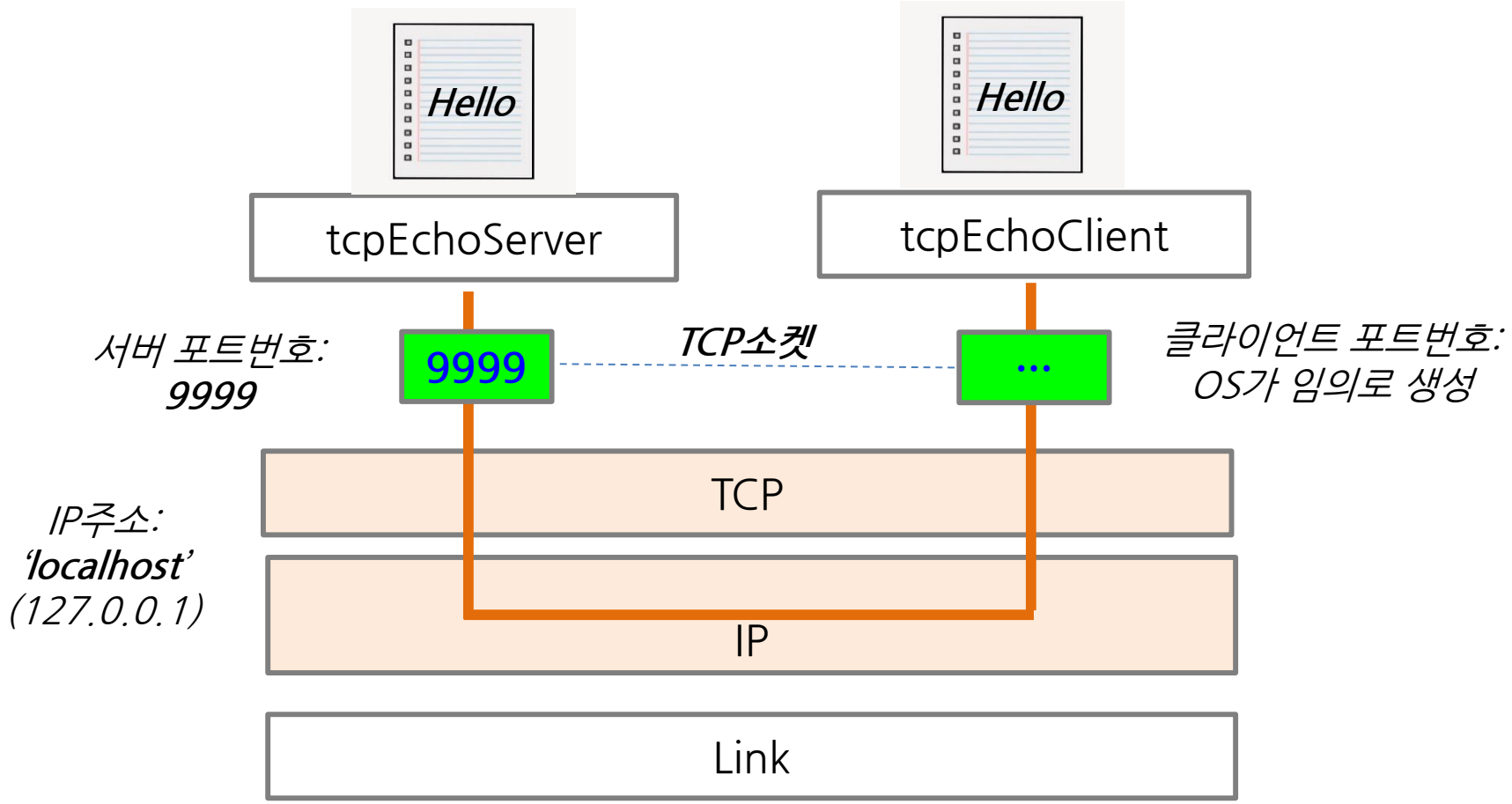
① 소켓생성 ② 연결 설정 ③ 데이터 송수신 ④ 연결해제 순으로 수행



** 파이썬에서 소켓으로 메시지 보낼 때는 기존 send( ) 대신에 sendall( ) 사용 가능*
sendall( )은 파이썬 기반의 함수로 버퍼내 모든 데이터를 전송하는 기능제공

(실습) TCP 에코 서버 클라이언트 네트워킹 환경

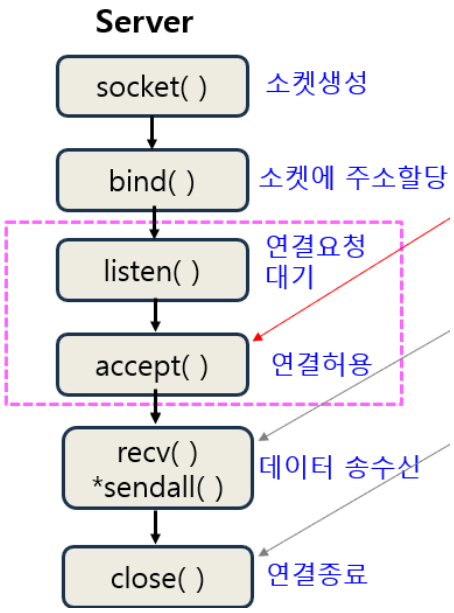
- IP주소는 'localhost' 사용:
  localhost는 loopback 주소로 내 컴퓨터 지칭하는 host 이름, 실제 IP주소는 127.0.0.1
  물리적인 링크를 사용하지 않고 IP단에서 바로 상위로 올려 보내 패킷을 외부로
  전송하지 않음



TCP 에코 서버 프로그램

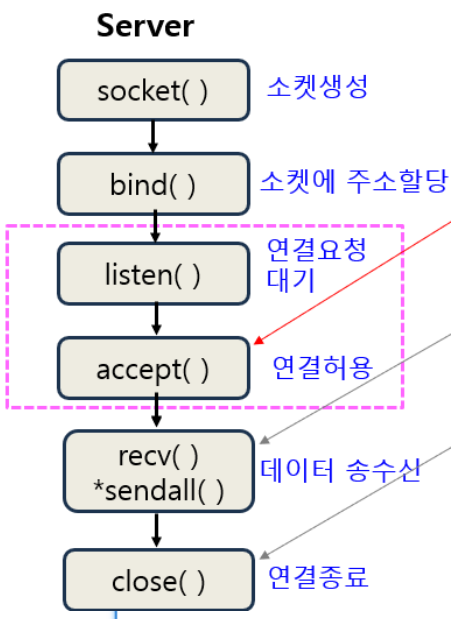
tcpEchoServer.py

```python
# TCP Echo server program

import socket

IP = 'localhost'  # loopback ip (127.0.0.1)
PORT = 9999
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Echo server start")

# socket() 서버 소켓 생성
# AF_INET:주소종류는 IPv4, SOCK_STREAM:통신타입은 TCP
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# (optional code) winError 10048 에러 해결을 위한 추가 코드
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)

# bind(주소정보) 소켓에 IP주소와 포트번호 지정
server.bind(address)
# listen() 클라이언트 접속요청 대기 상태
server.listen()
print(">> Echo server wait.........")

```



TCP 에코 서버 프로그램

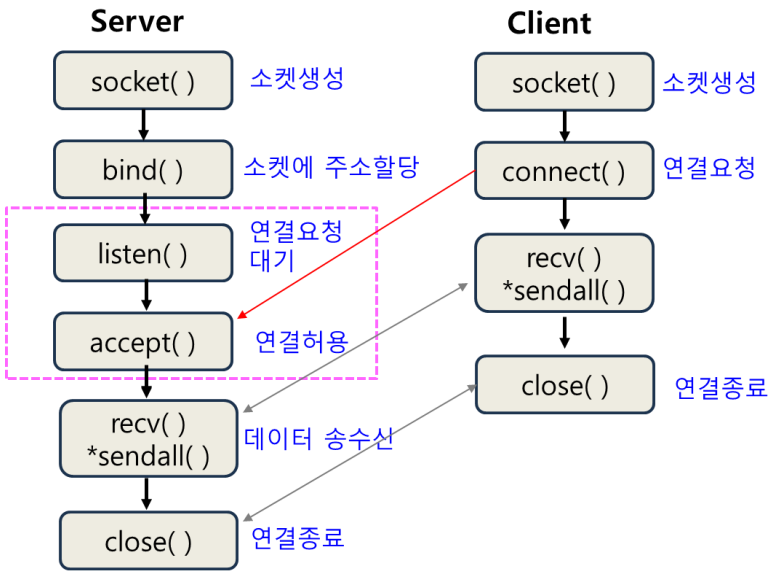
```python
# accept() 클라이언트 접속 요청이 있을 시 수락
# 통신소켓과 클라이언트 주소정보를 반환
conn, addr = server.accept()
print(f">> {addr} is connected")

# recv(받을 메시지크기) 통신소켓에 데이터가 들어올때 까지 기다림
# 통신소켓에 데이터가 들어오면 메시지 반환
rxmsg = conn.recv(1024)
msg = rxmsg.decode('utf-8')  # 받을때는 디코딩
print(">> recv msg:", msg)

# sendall(보낼데이터) 통신소켓에 메시지 보내기
# 에코서버는 받은 데이터를 그대로 돌려줌
conn.sendall(msg.encode('utf-8'))  # 보낼때는 인코딩

# close() 서버 소켓 종료
conn.close()      # 통신소켓 종료
server.close()    # 리스닝소켓 종료

print(">> Echo server End")
```



TCP 에코 클라이언트 프로그램

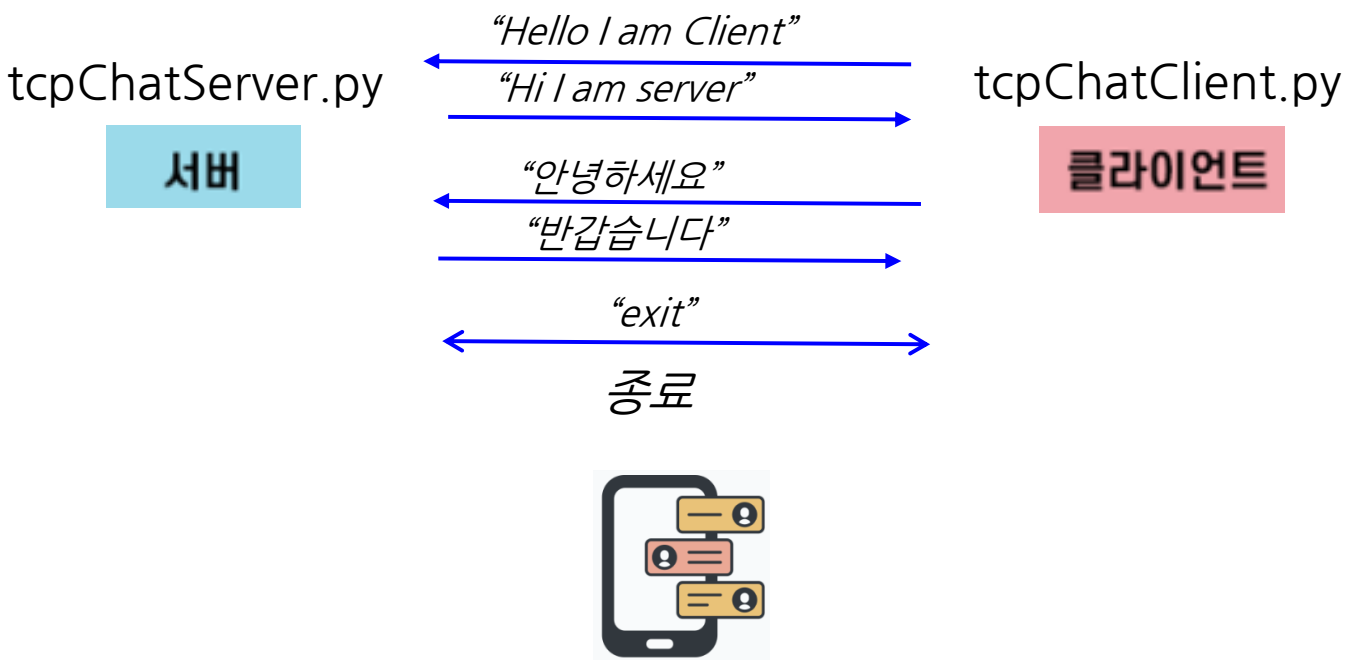
tcpEchoClient.py

```python
# TCP echo client program

import socket

IP = 'localhost'  # 서버 IP
PORT = 9999       # 서버 포트번호
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Echo client start")

# socket() 클라이언트 소켓 생성 (ipv4, tcp 소켓)
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# connect(서버주소정보) 서버소켓으로 연결요청
client.connect(address)
print(">> Connect server")

# sendall(보낼데이터) 서버로 메시지 보내기
msg = input(">> send msg: ")
client.sendall(msg.encode('utf-8')) # 보낼 때 인코딩

# recv(받을 메시지크기) 에코서버로부터 되돌아 온 메시지 수신
rxmsg = client.recv(1024)
msg = rxmsg.decode('utf-8')   # 받을때는 디코딩
print(">> recv msg: ", msg)

# close() 클라이언트 소켓 종료
client.close()
print(">> Client End")
```



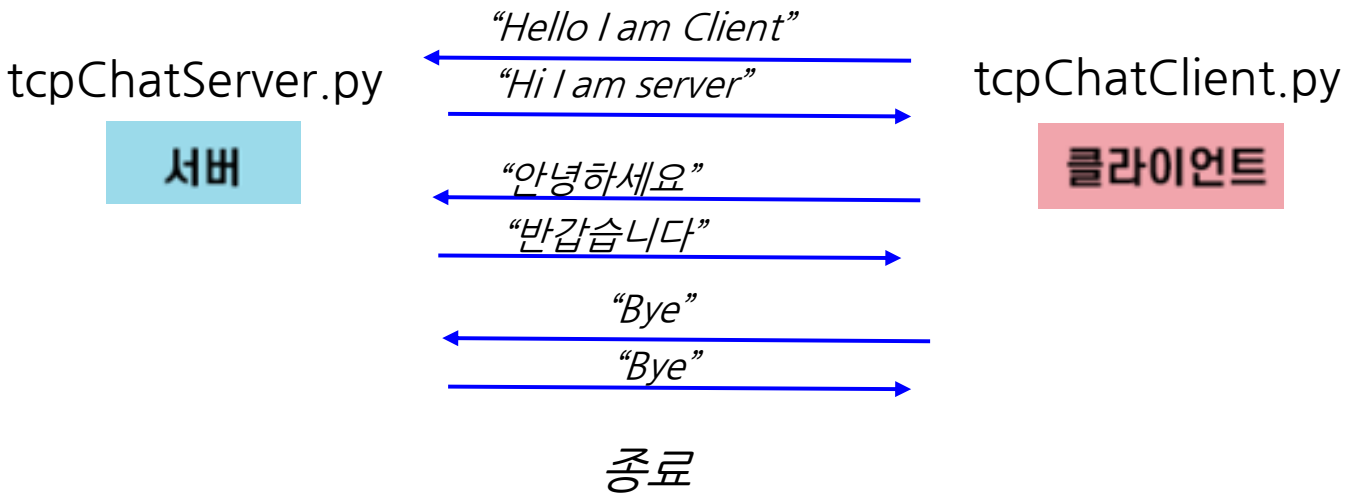
1. TCP 소켓 프로그램 응용 실습1 - 1:1 채팅

- TCP 소켓을 이용한 서버 클라이언트간 1:1 채팅 프로그램 구현하기



1. TCP 소켓 프로그램 응용 실습1 - 1:1 채팅

▪ TCP 소켓을 이용한 서버 클라이언트간 1:1 채팅 구현조건



1:1 채팅 구현 시 반영할 사항

조건1) 클라이언트가 먼저 메시지를 전송하고 서버가 응답하는 방식
조건2) 서버 클라이언트간 연결된 상태에서는 채팅은 지속할 수 있어야 한다
조건3) 서버 혹은 클라이언트가 'exit' 를 입력하면 연결은 정상 종료되어야 한다

1. TCP 소켓 프로그램 응용 실습1 - 1:1 채팅

- **TCP 소켓을 이용한 서버 클라이언트간 1:1 채팅 동작방식**

tcpChatServer.py

서버

```
>> Server start
>> Server wait.........
>> ('127.0.0.1', 54819) is connected
Client >> Hello server
Server >>  Hi client
Client >> 안녕하세요
Server >>  반갑습니다
Client >> Bye
Server >>  Bye bye
Disconnect by Client
>> Server End
```

tcpChatClient.py

클라이언트

```
>> Client start
>> Connect server
Client >> Hello server
Sever >> Hi client
Client >> 안녕하세요
Sever >> 반갑습니다
Client >> Bye
Sever >> Bye bye
Client >> exit
>> Exit by User
Client End
```

Client에서 'exit' 입력 시

```
Server >>  exit
Exit by User
>> Server End
```

Server에서 'exit' 입력 시

```
Disconnect by Server
Client End
```

1. TCP 소켓 프로그램 응용 실습1 - 1:1 채팅

- TCP 소켓을 이용한 서버 클라이언트간 1:1 채팅 구현하기

(실습1) TCP ECHO 서버와 클라이언트 프로그램을 기초로 하여

서버와 클라이언트간 1:1 채팅을 구현해 봅시다.

tcpEchoServer.py → tcpChatServer.py로

tcpEchoClient.py → tcpChatClient.py로 구현

*원본은 유지하고 다른 이름으로 저장 후 구현하기

2. TCP 소켓 응용 실습2 - 숫자 게임

- **TCP 소켓을 이용한 서버 클라이언트간 숫자 맞추기 게임 구현하기**

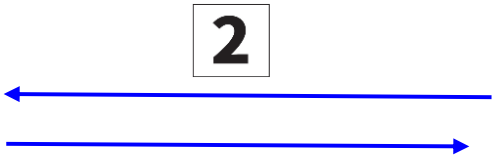
TCP 채팅 서버와 클라이언트 파일을 응용하여
서버와 클라이언트간 숫자를 맞추는 통신 게임을 구현해 봅시다“

작동방식: “서버는 1~9 사이의 한 숫자를 무작위로 key 값을 선정하고
클라이언트가 TCP 소켓을 이용해 접속하여 그 숫자를 맞추는 게임 “

tcpGameServer.py

서버

Key



tcpGameClient.py

클라이언트

2. TCP 소켓 응용 실습2 - 숫자 게임

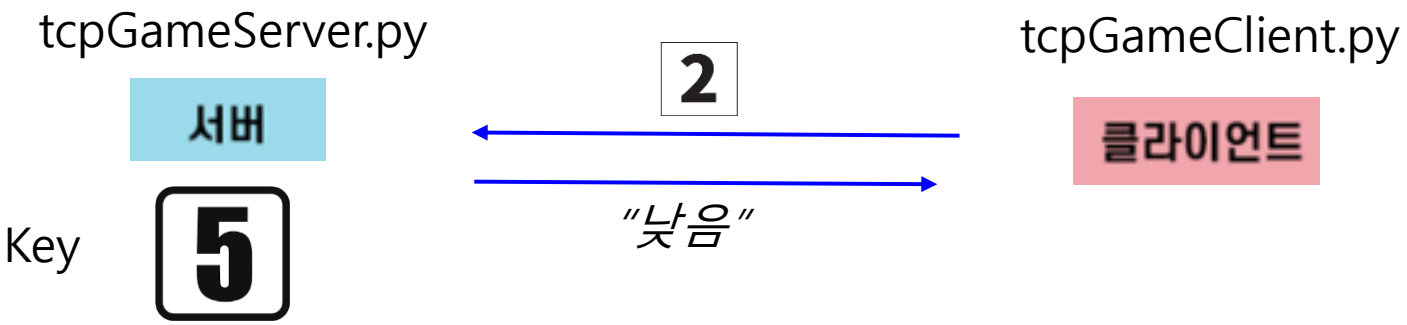
- **TCP 소켓을 이용한 서버 클라이언트간 숫자 맞추기 게임 구현하기**

작동방식: "서버는 1~9 사이의 한 숫자를 무작위로 key 값을 선정하고
클라이언트가 TCP 소켓을 이용해 접속하여 그 숫자를 맞추는 게임 "

* 서버와 클라이언트간 통신은 TCP 사용, 서버포트는 9090 사용

1) 서버는 1 ~ 9 사이의 숫자 하나를 무작위로 Key를 생성하고 클라이언트를 기다린다
2) 클라이언트는 서버에 접속하여 1~9 사이의 숫자를 전송하여 Key값을 맞추는 게임
3) 서버는 클라이언트로 받은 숫자가 Key값보다 높을 때는 "높음" ,
   낮을 때는 "낮음 " 을 전송
4) 서버가 클라이언트로 key 값을 받으면 "정답 " 이라고 응답하고
   서버와 클라이언트 연결 종료
5) 클라이언트가 중간에 '0'를 입력하면 게임 종료
6) 서버도 클라이언트로부터 '0' 을 수신하면 게임종료

주의) 숫자가 아닌 문자가 입력되면 "입력값이 잘못되었습니다" 출력하고 재입력을 기다림



2. TCP 소켓 응용 실습2 - 숫자 게임

- **TCP 소켓을 이용한 서버 클라이언트간 숫자 맞추기 게임 동작예시**

tcpGameServer.py

서버

Key 8

1

"낮음"

tcpGameClient.py

클라이언트

```
>> Game Server start
>> Game Server wait.............
>> ('127.0.0.1', 55496) is connected
>> Server key value is 8
>> recv number:  1
>> recv number:  5
>> recv number:  9
>> recv number:  8
>> Game Server End
```

```
>> Connect Game Server
1-9 사이의 숫자를 입력하세요(0은 게임포기): 1
Answer >> 낮음
1-9 사이의 숫자를 입력하세요(0은 게임포기): 5
Answer >> 낮음
1-9 사이의 숫자를 입력하세요(0은 게임포기): 9
Answer >> 높음
1-9 사이의 숫자를 입력하세요(0은 게임포기): 8
Answer >> 정답
>> Client End
```

문자가 입력된 경우

```
>> Connect Game Server
1-9 사이의 숫자를 입력하세요(0은 게임포기): abc
입력값이 잘못되었습니다.
1-9 사이의 숫자를 입력하세요(0은 게임포기):
```

```
>> recv number:  0
Disconnect by Client
>> Game Server End
```

0을 입력한 경우

```
1-9 사이의 숫자를 입력하세요(0은 게임포기): 0
Answer >>
>> Client End
```

2. TCP 소켓 응용 실습2 - 숫자 게임

- TCP 소켓을 이용한 서버 클라이언트간 숫자 맞추기 게임 구현하기

(실습2) TCP 채팅 서버와 클라이언트 프로그램을 기초로 하여

서버와 클라이언트간 숫자 게임을 구현해 봅시다.

tcpChatServer.py → tcpGameServer.py로

tcpChatClient.py → tcpGameClient.py로 구현

* 원본은 유지하고 다른 이름으로 저장 후 구현하기

3. TCP 소켓 응용 실습3 - 서로 다른 컴퓨터와 통신하기

(실습3) 네트워크를 통해 서로 다른 컴퓨터와 숫자 게임하기

- 이제껏 localhost(127.0.0.1) loopback 주소를 가지고 내컴퓨터 내에서 실습을 진행했습니다.
- 실습 환경을 바꾸어 실제 네트워크 상에서 다른 컴퓨터와 메시지를 주고 받아봅시다

- 조건변경1: 서버 포트번호는 1234로 지정
- 조건변경2: 서버는 1~1000사이의 랜덤한 정수를 key값으로 합니다.

- 옆 사람(다른 컴퓨터)와 짝을 지어 한 사람은 서버, 다른 사람은 클라이언트가 되어 숫자 게임을 진행해 봅시다. 단, 정답을 맞출 때까지 진행합니다.

- 역할을 바꾸어 진행해 봅시다

4. 지도 앱 제작하기

❑ 지도 앱

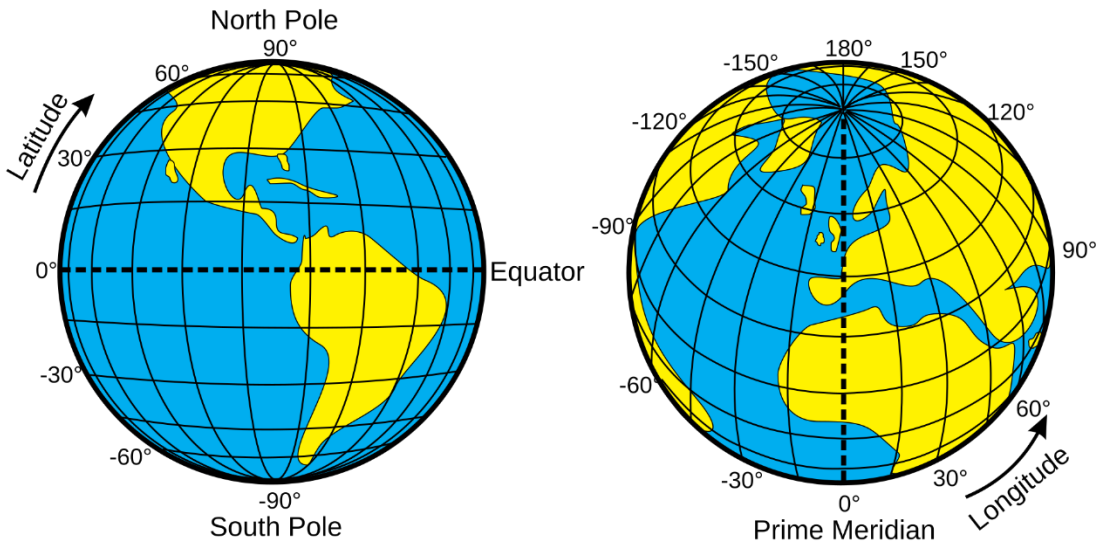
- 앱에서 제공하는 기능
  - 내 위치 찾기 기능
  - 내 위치 공유하기 기능

- 지도 앱 제작 방법
  1) [지도(map)] 컴포넌트 이용 :
     앱 인벤터에서 제공하는 지도 컴포넌트 이용
  2) [웹뷰어] 컴포넌트 이용:
     앱에서 지도 웹 사이트(구글 지도 혹은 네이버 지도)를 이용
  3) [액티비티스타터] 컴포넌트 이용:
     다른 앱을 호출하여 사용

4. 지도 앱 제작하기

❑ 지도는 위도와 경도로 위치 표시

- 위도(latitude) : 적도를 기준으로 북쪽 및 남쪽으로 얼마나 떨어져 있는지 나타내는 위치
- 경도(longitude) : 본초 자오선을 기준으로 동쪽 또는 서쪽으로 얼마나 떨어져 있는지 나타내는 위치



그림출처: 위키백과

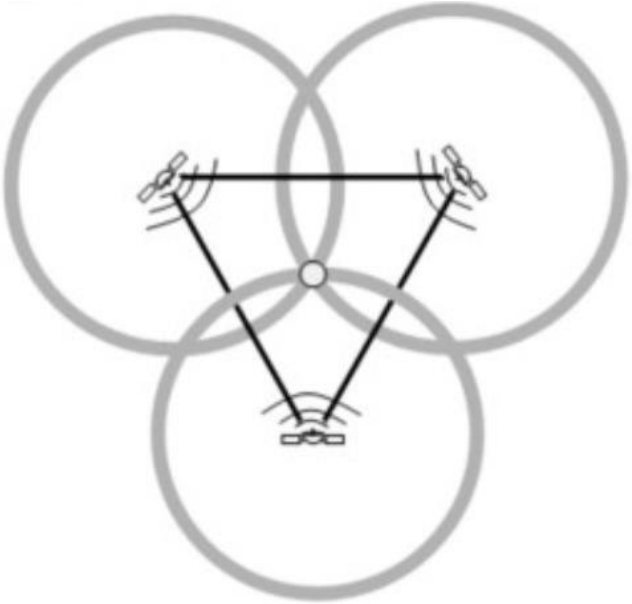
예) 서울은 북위 37.5642135° 동경 127.0016985°

4. 지도 앱 제작하기

❑ 앱 인벤터 [위치센서]

- 지도에서 위치를 표시할 수 있게 하는 컴포넌트
- GPS로부터 신호를 수신해 사용자의 현재 위도, 경도, 고도, 주소등의 정보 제공
- 최소 3GPS로 부터 신호를 수신하여 위치를 계산

**GPS(Global Positioning System)는 위성을 이용해 위치, 항법, 시작 정보를 제공하는 시스템*



4. 지도 앱 제작하기

1) 새 프로젝트 만들기

- ❑ **프로젝트 이름: map**

4. 지도 앱 제작하기 - 디자인

2) 앱 디자인하기

※ Screen1 속성 >> 수평- 가운데, 제목 – 위치찾기 (제작:이름)

	컴포넌트	이름	속성
①	레이블	레이블1_내위치	글꼴굵게, 글꼴크기-15, 너비 – 부모요소 텍스트- 내 위치 찾는 중..... 정렬 – 가운데
②	수평배치1	수평배치1	수평, 수직정렬 – 가운데, 너비-부모요소
	버튼	버튼1_내위치	배경색-초록, 글꼴굵게, 글꼴크기-15, 모양-직사각형, 텍스트- 내위치
	버튼	버튼1_위치공유	배경색-주황, 글꼴굵게, 글꼴크기-15, 모양-직사각형, 텍스트- 내위치
③	*지도1	지도1	높이-부모, 너비- 부모, 나침반보이기
④	*마커1	마커1	
⑤	위치센서1	위치센서1	
⑥	*공유		

* 팔레트 >> 지도 >> 지도, 마커
* 팔레트 >> 소셜 >> 공유

4. 지도 앱 제작하기 - 코딩

3) 앱 코딩하기 : [위치센서]로 내 위치 찾아 지도에 표시하기

- 내 위치가 변경될 때 마다 자동으로 [위치센서]로 좌표값 받아서 지도에 표시

4. 지도 앱 제작하기 - 코딩

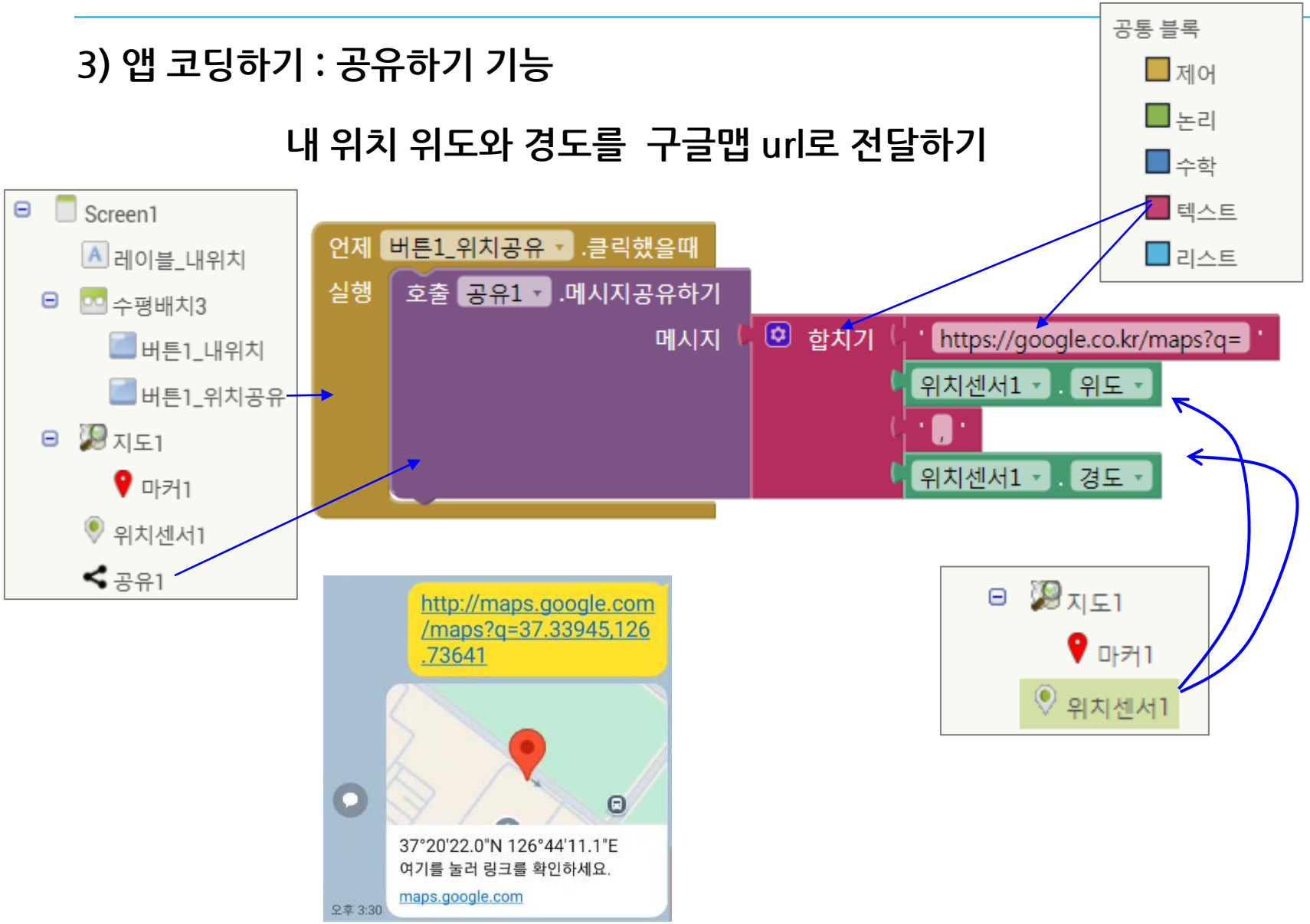
3) 앱 코딩하기 : [버튼_내위치]를 클릭했을 때 내 위치를 지도에 표시하기

- 자동으로 지도에 나타나지 않을 경우 [버튼_내위치]를 통해 지도에 나타내기

4. 지도 앱 제작하기 - 코딩

3) 앱 코딩하기 : 공유하기 기능

내 위치 위도와 경도를 구글맵 url로 전달하기



참조) 구글맵 위치 표시하기

- 구글맵에 위도와 경도를 URL로 보내면 지도에 위치를 표시할 수 있음

구글맵의 URL 구조

https://www.google.co.kr/maps?q= 37.338751, 126.736160

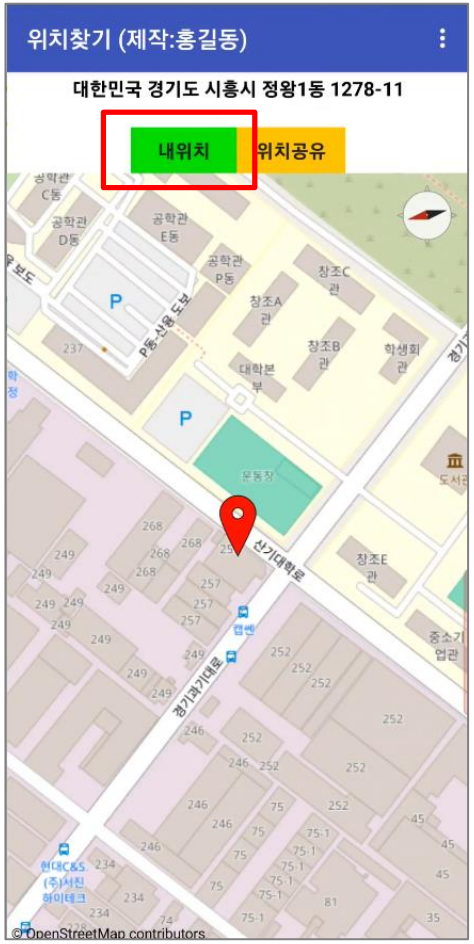
위도 *경도*

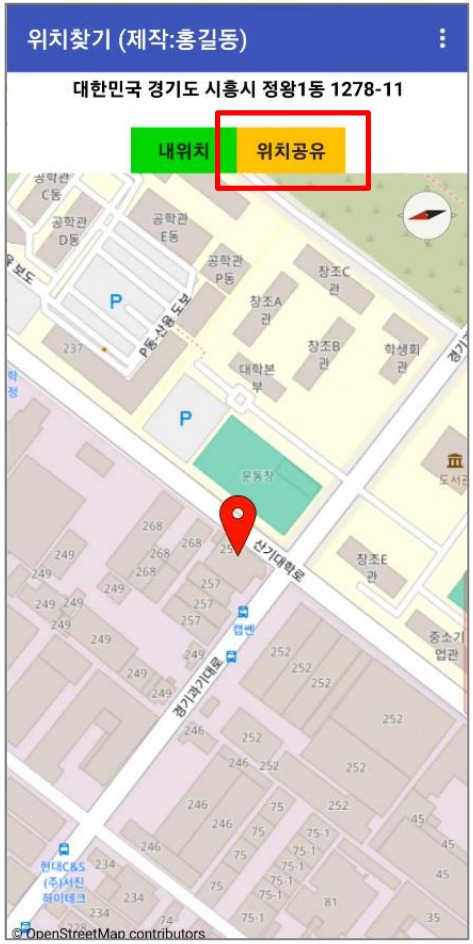
https://www.google.co.kr/maps?q=37.338768, 126.73607418

4. 지도 앱 제작하기 - 실행

4) 앱 실행하기

- **[내위치 버튼]**을 클릭하면 내 위치의 주소와 지도에 표시되는지 확인
- **[위치공유 버튼]**을 클릭하면 소셜(문자, 톡등) 앱이 작동하고 하나를 선택하여 전송





3주차 실습과제 제출 안내

❑ 3주차 과제 제출 안내

- 과제: 3주차에 실습한 TCP 소켓을 이용한
  서버와 클라이언트간 1:1 채팅프로그램 파일 제출하기

- 파일명: tcpChatServer_이름.py , tcpChatClient_이름.py

- 실습 시 잘 안된 부분 혹은 이해가 안되는 부분은 comment 남겨 주세요

Thank You